Project 2: Cache & Memory Performance Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 16, 2026

Abstract

[**TODO:** Two to four sentences: per-level latencies measured, DRAM bandwidth, where the throughput–latency knee sits and why, and the measured cache/TLB-miss impact.]

1 Introduction

[**TODO:** Motivation: the L1/L2/L3/DRAM hierarchy on this machine (sizes, expected latencies) and why locality, concurrency and page size decide real performance.]

This report answers:

1. What is the zero-queue access latency of each level, and the DRAM bandwidth under different patterns, granularities and read/write mixes? (Sections 4 to 6)
2. Where does added concurrency stop buying throughput, and does Little's Law hold? (Section 7)
3. How do working-set size, cache misses and TLB misses show up in runtime and counters? (Sections 8 to 10)
4. Which benchmark mistake produces misleading numbers? (Section 11)

[**TODO:** One-paragraph summary of the main findings.]

2 Methodology

2.1 Platform

2.2 Tools

`mem_latency`	single-thread access latency for a working set of a given size
`mem_bandwidth`	bandwidth and sampled per-operation latency with T pinned threads
`kernel_miss`	simple loop kernel under `perf stat` for cache/TLB correlation
Intel MLC	[**TODO:** version and options, if used [1]]
perf	not available (not installed) [2]

Table 1: Measurement environment.

CPU	Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64)
SIMD ISA	sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm)
Topology	1 socket(s), 10 cores/socket, 2 threads/core (20 logical); NUMA: 1 node(s)
SMT	on (2 threads/core)
Caches	L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance)
Memory	63 GiB total; DIMMs: not available (needs root/dmidecode)
Pages	base 4096 bytes; THP: always
OS / kernel	Arch Linux; 7.2.9-arch1-1
Frequency policy	governor: powersave; driver: intel_pstate; turbo: enabled; EPP: balance_performance
Pinning / isolation	[**TODO:** e.g. `taskset -c 2` on a P-core; isolcpus if used]; no isolcpus; runs pinned with taskset/–cpu
Compilers	gcc (GCC) 16.2.1 20260810; clang version 23.1.1
Tools	not available (not installed) (perf_event_paranoid 2); not available (not installed); MLC: not available (not installed)
GPU (optional)	NVIDIA GeForce RTX 3070; CUDA not available (no CUDA)
Memory configuration	[**TODO:** DIMM count, capacity, rated vs. configured speed, channels]
Collected	2026-10-12 18:35 EDT

2.3 Controls

- Pinning and isolation: [**TODO:** CPUs used, one logical CPU per physical core, P-cores only?]
- Frequency: governor powersave, turbo enabled. [**TODO:** justify / what was changed]
- Pre-faulting and warm-up: [**TODO:** how buffers are faulted in and warmed before timing]
- Repetitions: ?? per point; median with IQR error bars. [**TODO:** justify]
- Page size: base 4096 bytes; transparent huge pages: always. [**TODO:** how huge-page backing was verified]
- Background load: [**TODO:** how the machine was kept quiet]

3 Validation and Sanity Checks

[**TODO:** Checks that the tools measure what they claim, e.g.:]

- [**TODO:** L1 latency on the order of a few CPU cycles at the measured frequency.]
- [**TODO:** Single-thread sequential bandwidth below the theoretical DIMM/channel limit.]
- [**TODO:** Cross-check against Intel MLC or another reference for at least one configuration.]
- [**TODO:** Timer and loop overhead measured with an empty access loop and subtracted or bounded.]

Table 2: Zero-queue access latency per level. Cycles assume 1.0 GHz [**TODO:** measured frequency].

Data pending: `data/zero_queue.csv`
run the experiment in ACS-Code, then `scripts/sync_data.sh`

4 Zero-Queue Baselines

Hypothesis (written before measuring)

[**TODO:** Expected latency per level (in ns and in CPU cycles) and why.]

Fixed	one thread pinned to CPU ??; random reads; granularity ?? bytes
Varied	working-set size placed in L1, L2, LLC, DRAM
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh zero_queue`

4.1 Analysis

[**TODO:** Which level each size actually lands in (evidence), caveats (timer/loop overhead, TLB, prefetchers), and comparison with published or MLC numbers.]

5 Access Pattern and Granularity

Hypothesis (written before measuring)

[**TODO:** How sequential vs. random access and 64/256/1024-byte granularity should affect bandwidth and latency, and why (prefetching, TLB reach, memcpy cost).]

Fixed	1 thread; ?? GiB DRAM-resident region; 100% reads
Varied	pattern (sequential, random); granularity (64, 256, 1024 bytes)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh pattern`

5.1 Analysis

[**TODO:** Explain the gap between sequential and random access and its dependence on granularity.]

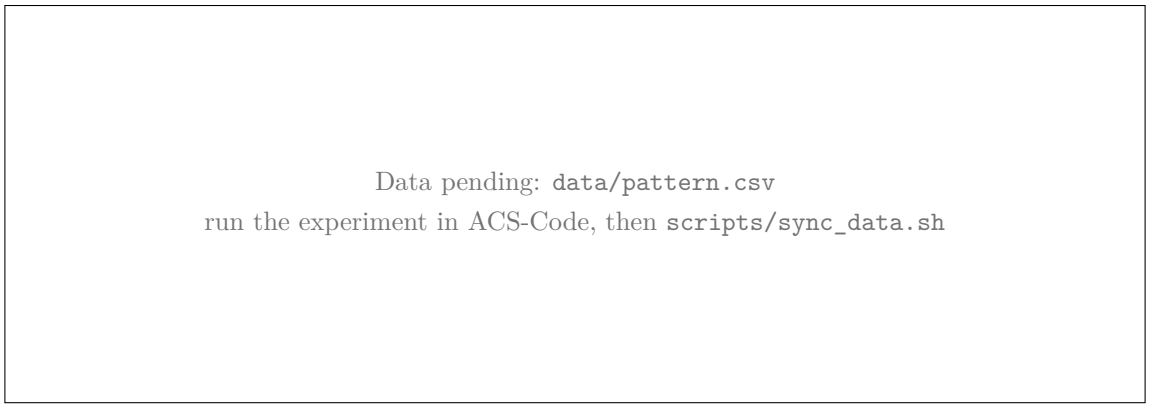


Figure 1: Access pattern and granularity. [**TODO:** takeaway]

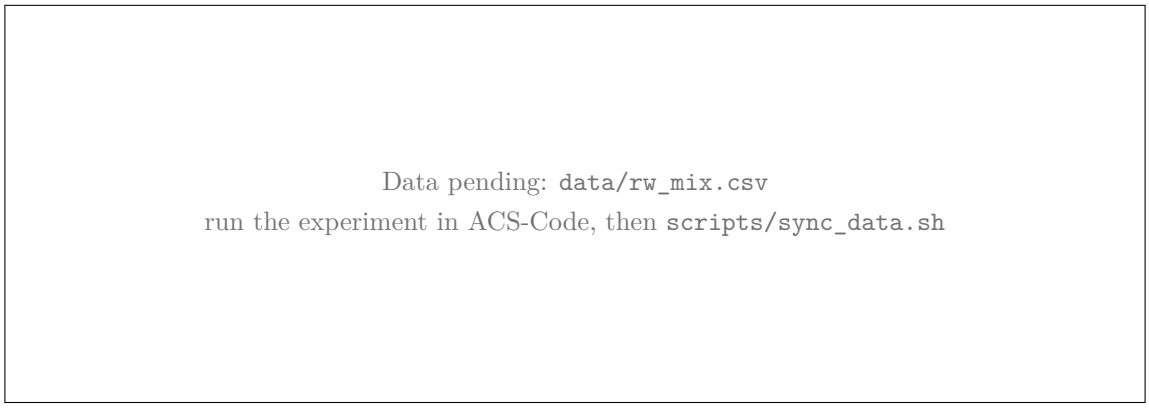


Figure 2: Bandwidth across read/write mixes. [**TODO:** takeaway; add latency]

6 Read/Write Mix

Hypothesis (written before measuring)

[**TODO:** Expected throughput and latency for 100% read, 100% write, 70/30 and 50/50, and why writes differ (read-for-ownership, cache pollution, write-combining).]

Fixed	1 thread; random access; granularity ?? bytes; region ??
Varied	read percentage: 100, 70, 50, 0
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh rw_mix`

6.1 Analysis

[**TODO:** Compare throughput and latency across mixes and explain the differences.]

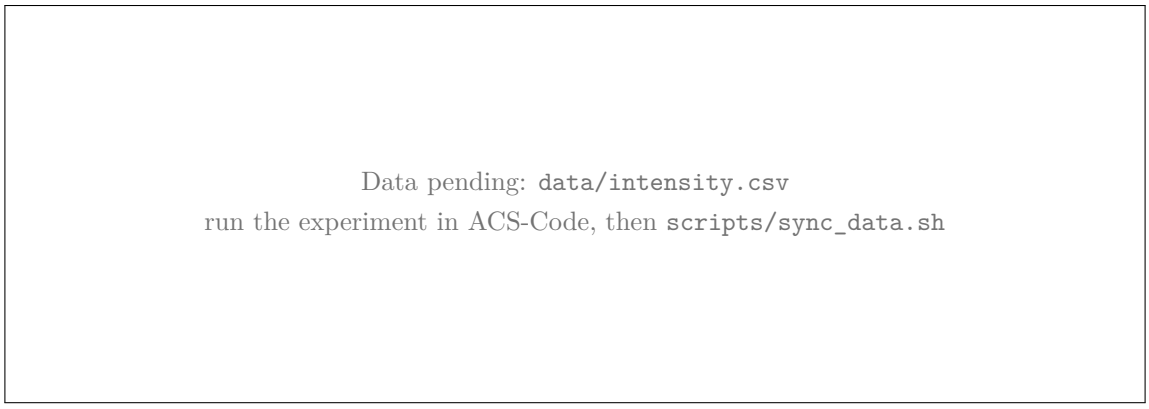
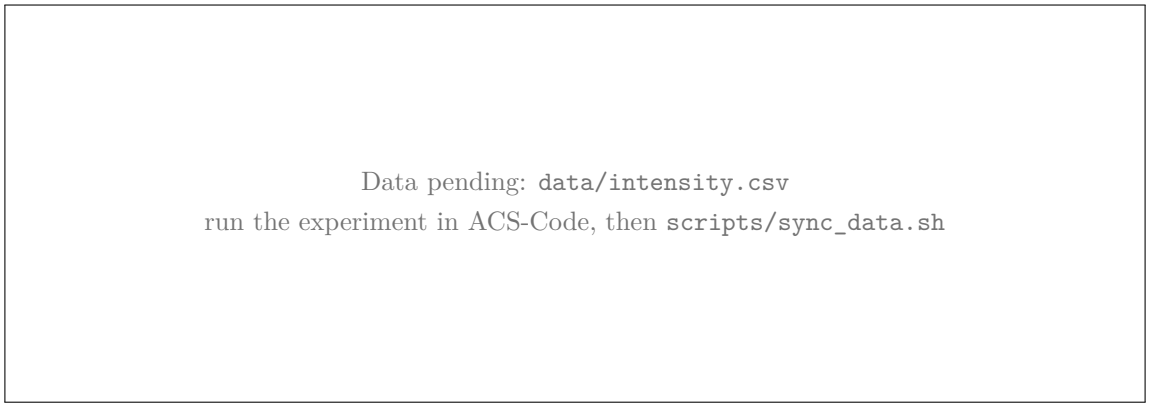


Figure 3: Throughput vs. latency; labels give the thread count. [**TODO:** mark the knee]

Table 3: Little's Law: measured λW against the number of busy threads.



7 Intensity Sweep and the Throughput–Latency Knee

Hypothesis (written before measuring)

[**TODO:** Thread count at which throughput should stop scaling and latency should rise, and why (cores, memory channels, shared bandwidth).]

Fixed	region ?? per thread; random reads; granularity ?? bytes
Varied	threads $T = 1, 2, 4, \ldots$ (pinned: [**TODO:** CPU list])
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh intensity`

7.1 Little's Law check

With T threads always busy, the number of requests in the system is $L = T$, so throughput λ and mean latency W should satisfy $L = \lambda W$ [3].

7.2 Analysis

[**TODO:** Where the knee is and why; diminishing returns beyond it; whether $\lambda W \approx T$ holds and what a deviation means (sampling, think time, oversubscription).]

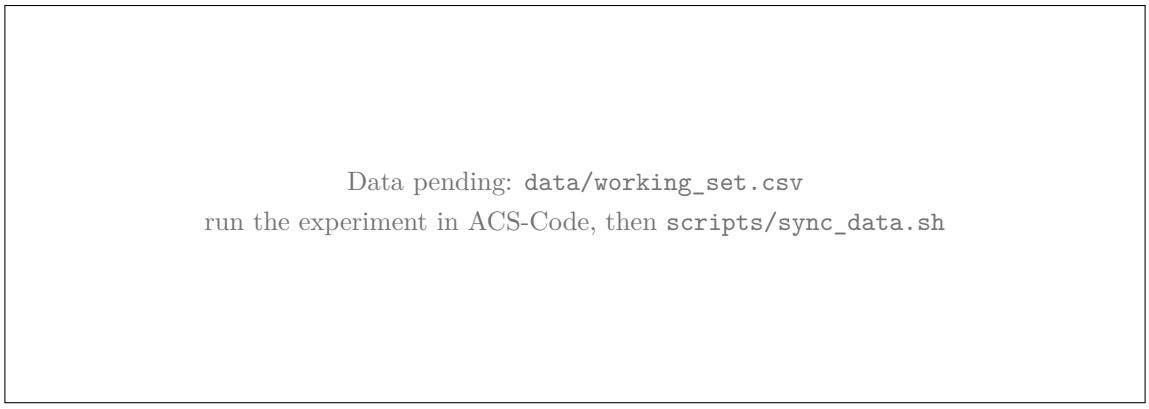


Figure 4: Access latency across working-set sizes. [**TODO:** takeaway]

8 Working-Set Sweep

Hypothesis (written before measuring)

[**TODO:** Where latency steps should appear given L1d = ??, L2 = ??, LLC = ?? on this CPU.]

Fixed	1 thread pinned; random reads; granularity ?? bytes; base pages
Varied	working-set size from 4 KiB to ?? GiB
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh working_set`

8.1 Analysis

[**TODO:** Transition regions, and how sharp or gradual they are and why (shared LLC, prefetchers, TLB reach).]

9 Cache-Miss Impact

Hypothesis (written before measuring)

[**TODO:** How runtime per element should track cache misses as footprint and traversal order change.]

Kernel	$z_i = x_i \cdot y_i$ (float64)
Varied	N (footprint); traversal order (sequential, random permutation)
Counters	[**TODO:** events used; note that `perf stat` covers the whole process]
Script	`PERF=1 Cache_and_Memory_Performance_Profiling/scripts/run.sh cache_miss`

9.1 AMAT interpretation

$$\mathrm{AMAT} = t_{L1} + m_{L1}\left(t_{L2} + m_{L2}\left(t_{L3} + m_{L3}\, t_{\mathrm{DRAM}}\right)\right) \tag{1}$$

[**TODO:** Plug in the zero-queue latencies (Table 2) and measured local miss ratios; compare the predicted and measured time per element.]

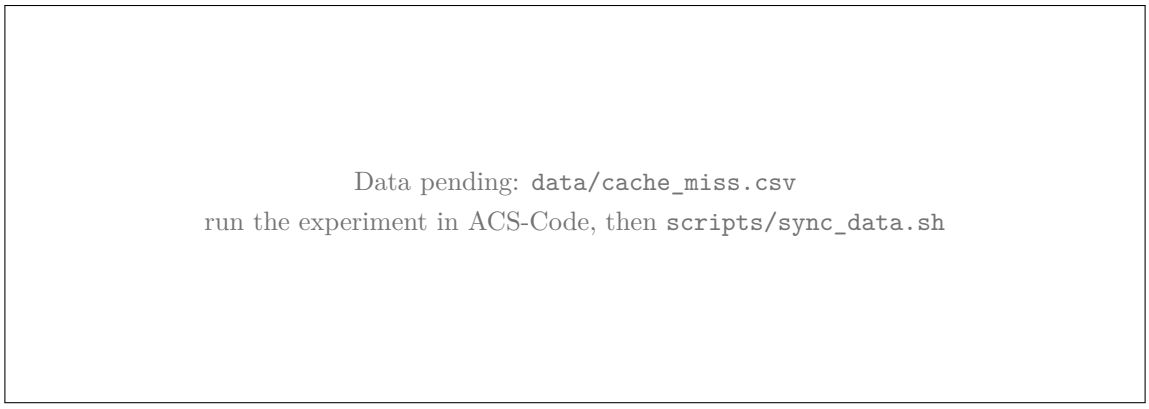


Figure 5: Runtime per element versus footprint and order. [**TODO:** takeaway]



Figure 6: Runtime with base vs. huge pages. [**TODO:** takeaway; add dTLB-load-misses]

10 TLB-Miss Impact

Hypothesis (written before measuring)

[**TODO:** Expected effect of 2 MiB pages on dTLB misses and runtime for random access over large footprints, given the TLB sizes of this CPU.]

Kernel	$z_i = x_i \cdot y_i$ (float64), random traversal
Varied	page size (base, transparent huge pages); N
Verification	[**TODO:** how huge-page backing was confirmed (AnonHugePages)]
Script	`PERF=1 Cache_and_Memory_Performance_Profiling/scripts/run.sh tlb`

10.1 Analysis

[**TODO:** TLB reach (entries × page size) vs. footprint; change in dTLB misses and page walks; runtime effect and its limits.]

11 Benchmark Pitfall

Hypothesis (written before measuring)

[**TODO:** What the flawed setup will report and why it looks plausible.]

Pitfall	[**TODO:** e.g. measuring cache reuse instead of DRAM, unpinned threads, frequency scaling]
Wrong setup	[**TODO:**]
Right setup	[**TODO:**]
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh pitfall`

[**TODO:** Figure or table comparing the two setups.]

11.1 Explanation

[**TODO:** Why the flawed number is wrong, how to detect it, and how the methodology avoids it.]

12 Anomalies and Limitations

12.1 Unexpected results

- [**TODO:** Observation → plausible hypothesis → how it was (or could be) tested.]

12.2 Limitations

- [**TODO:** e.g. single machine and memory configuration, sampled latency resolution, counters covering initialization, THP backing not guaranteed.]

13 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used

[**TODO:** Tools and models, and for which parts of the project.]

What they helped generate

[**TODO:** e.g. repository scaffolding, build and run scripts, debugging help.]

What was independently modified, tested, or rejected

[**TODO:** Concrete examples, including at least one AI suggestion that was wrong or misleading and how it was caught.]

Validation

[**TODO:** How measurements were checked independently (sanity checks, MLC cross-check, counters).]

14 Conclusion

[**TODO:** Three to five sentences: the latency and bandwidth picture of this hierarchy, where concurrency stops paying off, and the most important lesson about locality, cache and TLB behavior.]

References

[1] Intel Corporation, *Intel memory latency checker*, TODO: version and download URL.

[2] *Perf: Linux profiling with performance counters*. [Online]. Available: `https://perf.wiki.kernel.org/`

[3] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. DOI: `10.1287/opre.9.3.383`

A Reproducibility

Code, scripts and raw data: `github.com/SKempiak/ACS-Code`, commit [**TODO:** hash].

```
git clone https://github.com/SKempiak/ACS-Code.git && cd ACS-Code
git checkout <commit>
make && make test
scripts/collect_env.sh
ACS_CPU=2 ACS_CPUS=2,4,6,8 PERF=1 Cache_and_Memory_Performance_Profiling/scripts/run.sh all
scripts/sync_data.sh Cache_and_Memory_Performance_Profiling
```

Table 4: Raw data behind each figure.

Figure / table	Raw run directory (`Cache_and_Memory_Performance_Profiling/data/raw/...`)
Table 2	[**TODO:** `zero_queue/<timestamp>`]
Figure 1	[**TODO:** `pattern/<timestamp>`]
Figure 2	[**TODO:** `rw_mix/<timestamp>`]
Figure 3, Table 3	[**TODO:** `intensity/<timestamp>`]
Figure 4	[**TODO:** `working_set/<timestamp>`]
Figure 5	[**TODO:** `cache_miss/<timestamp>`]
Figure 6	[**TODO:** `tlb/<timestamp>`]